Implementation of a 10BASE-T1L Interface for ESP32 Using ADIN1110

[*]Project conducted within the scope of "Schaltungstechnik 2"

Kilian Halbritter
Elektro- und Informationstechnik
TH Aschaffenburg
Aschaffenburg, Deutschland
s220419@th-ab.de

Benedikt Kotas
Elektro- und Informationstechnik
TH Aschaffenburg
Aschaffenburg, Deutschland
s220151@th-ab.de

Franz Marschall
Elektro- und Informationstechnik
TH Aschaffenburg
Aschaffenburg, Deutschland
s220273@th-ab.de

***Abstract*—The rise of Industry 4.0 requires robust, long-range communication protocols that extend beyond the capabilities of standard Wi-Fi or Bluetooth. Single Pair Ethernet (SPE), specifically the 10BASE-T1L standard, offers a solution with cable reaches up to 1000 meters. This paper details the design and implementation of a hardware interface connecting an ESP32 microcontroller to the Analog Devices ADIN1110 MAC-PHY. We present a custom PCB design utilizing a galvanic isolation transformer to ensure safety against Power over Ethernet (PoE) faults. Furthermore, a custom C++ driver architecture for the SPI protocol is proposed. Initial validation demonstrates correct power sequencing, although full SPI command verification remains a subject of ongoing debugging.**

***Index Terms*—ESP32, ADIN1110, 10BASE-T1L, Single Pair Ethernet, SPI, Industrial IoT**

I. Introduction

In the landscape of Industrial Internet of Things (IIoT), the "last mile" connectivity for sensors often poses a challenge. While wireless solutions like the ESP32 are versatile, they suffer from range limitations and susceptibility to interference in industrial environments. Standard Ethernet requires four pairs of wires and complex cabling.

The IEEE 802.3cg (10BASE-T1L) standard addresses this by enabling 10 Mbps Ethernet over a single twisted pair with a reach of up to 1 km. This project aims to bridge the gap between low-cost microcontrollers and industrial SPE by integrating the Espressif ESP32 with the ADIN1110 MAC-PHY. The primary contribution of this work is the documentation of the hardware coupling strategy and the development of a platform-independent C++ driver structure.

II. System Architecture

A. Hardware Design

The system creates a bridge between the wireless capabilities of the ESP32 and the wired reliability of SPE.

1) Microcontroller Unit (MCU): The host controller is an ESP32-DevKitC. It is responsible for the application logic and managing the network stack. The ESP32 is powered via USB (5V), which is regulated internally to 3.3V. This 3.3V rail is utilized to supply the peripheral PHY logic, ensuring signal level compatibility without the need for additional level shifters.

2) PHY Integration (ADIN1110): The ADIN1110 was selected for its ultra-low power consumption and integrated MAC, which offloads processing from the ESP32.

- **Clocking:** The ADIN1110 is clocked by a dedicated external 25 MHz crystal oscillator to ensure precise timing independent of the MCU clock.
- **Power Supply:** The PHY core requires 1.1V, which is generated internally or via a buck converter, while the VDDIO is supplied with 3.3V from the ESP32 to match the GPIO logic levels.
- **Safety Isolation:** A critical design decision was the inclusion of an isolation transformer on the MDI (Medium Dependent Interface) side. This ensures that if the device is connected to a network segment carrying Power over Ethernet (PoE), high voltages do not damage the low-voltage ESP32 circuitry.

III. Software Implementation

A. Development Environment

The firmware is developed using PlatformIO, leveraging the flexibility of the Arduino framework within a professional IDE structure. This allows for rapid prototyping while maintaining C++ class structures.

B. Driver Architecture

To abstract the hardware specifics, a custom driver class 'SPIDevice' was implemented. This class encapsulates the standard Arduino 'SPI.h' library, providing methods for transaction management.

The driver is designed to support the *Generic SPI* protocol mode of the ADIN1110. This mode was chosen over the OPEN Alliance SPI protocol for the initial implementation phase to reduce protocol overhead and complexity during debugging.

```
class SPIDevice {
  public:
  SPIDevice(int csPin, uint32_t clockSpeed,
  byte bitOrder, byte dataMode);
  void begin();
  void beginTransaction();
  void endTransaction();
  void select();
  void deselect();
  byte transfer(byte data);
  // Buffer operations for efficiency
  void readBuffer(byte* buffer, int length);
  void writeBuffer(const byte* buffer, int length);

  private:
  int _csPin;
  SPISettings _settings;
};
```

Listing 1. Header definition of the SPI abstraction layer

As shown in Listing 1, the class handles the Chip Select (CS) assertion and SPI settings configuration, ensuring that the ADIN1110 timing requirements are met before data transfer occurs.

C. Signal Measurement

For debugging and signal generation purposes, a PWM generation and frequency measurement routine was implemented on the ESP32 (utilizing GPIO 25 and 26). This subsystem serves to verify the ESP32's internal timing and interrupt handling capabilities, ensuring that the MCU is operating correctly before attempting high-speed SPI communication.

IV. Results and Validation

A. Hardware Verification

Initial power-up tests confirmed the integrity of the power supply design. The ADIN1110 status LEDs are active, indicating that the internal regulators are functioning and the 25 MHz clock is oscillating stably. The 3.3V logic levels between the ESP32 and ADIN1110 were verified using a multimeter.

B. Communication Challenges

Despite the electrical verification, the SPI communication is currently non-functional. The ESP32 successfully initiates SPI transactions (clock and MOSI signals active), but the ADIN1110 does not respond on the MISO line.

Potential causes currently under investigation include:

1) **Reset Timing:** The ADIN1110 requires a specific power-up reset sequence. The hardware reset pin might need a longer hold time controlled by the ESP32.
2) **SPI Mode Mismatch:** Although configured for Generic SPI, slight deviations in CPOL (Clock Polarity) or CPHA (Clock Phase) settings can prevent data latching.
3) **Protocol Header:** The generic SPI protocol requires a specific command header structure (Control + Address) which must be strictly adhered to.

V. Conclusion and Future Work

This paper presented the design of a 10BASE-T1L node based on the ESP32. The hardware integration, specifically the safety-oriented transformer coupling and shared power architecture, proved robust during initial electrical testing.

Future work will focus exclusively on the firmware layer. The immediate next step is to use a logic analyzer to inspect the SPI bus and verify the command frame structure against the ADIN1110 datasheet. Once the register access is established, the full TCP/IP stack will be integrated to benchmark the throughput over a 1 km cable.

Acknowledgment

We would like to thank the faculty of the "Schaltungstechnik 2" course for providing the resources and guidance to explore this advanced communication technology.

References

[1] IEEE Standard for Ethernet, "IEEE Std 802.3cg-2019," IEEE SA, 2019.
[2] Analog Devices, "ADIN1110 Datasheet: Robust, Industrial, Low Power 10BASE-T1L Ethernet MAC-PHY," 2021.
[3] Espressif Systems, "ESP32 Technical Reference Manual," V4.4, 2021.